

ROADMAP TÉCNICO

O que falta + como voltar a cada etapa — 2026-08-06
System v5.0

O que falta

Comandos exactos

Fases D2–D6

DATA · 2026-08-06

www.trinnityviseronsystem.io

TVS v5.0

0

Como voltar a cada etapa já feita (verificação rápida)

npm run lint

Typecheck global (tsc --noEmit)

npm run test

Todos os testes: core + web + omega + os

npm run demo

Demo operacional real (HTTP 9 endpoints)

npm run dev:web

API web standalone em dev (localhost:3000)

npm run call:start

CallSystemBridge standalone (chamadas)

npm run tvs

Estado do TVS OS (kernel/agentes/watchdog)

npm run build:exe:win

Reconstruir o .exe standalone (pkg)

npm run build:android

Build APK Android (expo run:android)

npm run build:apk-installer

Reconstruir o Setup.exe do APK (NSIS)

npm run pdfs:all

Regenerar TODOS os PDFs do projeto

1 Fase IA Cloud (bloqueada por chaves)

%, Adicionar ao .env e reiniciar: OPENAI_API_KEY, ANTHROPIC_API_KEY, GEMINI_API_KEY, XAI_API_KEY.

npm run dev

Reiniciar com as chaves carregadas

curl http://localhost:3000/api/ai/status

Ver o provider ativo e modelo

%, Ficheiros: src/web/jarvis/agent.ts (router de provider), src/web/revenue/routes.ts (/api/ai/status).

%, Depois: testar JARVIS complexo via POST /api/jarvis/chat e as análises de chamada (src/web/calls/learning.ts) passam a usar o modelo cloud.

2 Fase Chamadas telefónicas reais (próximo passo D2)

%, MISSING: Media Streams do Twilio (bidi stream de áudio) para resposta em voz em tempo real em vez de Gather por turno.

%, MISSING: verificar/destino de voz no número Twilio (trial bloqueia chamadas para números não validados).

%, MISSING: ligar a rececionista de empresas (D5) ao telefone — quando chega chamada inbound, responder com o knowledge base da empresa.

%, MISSING: registos — migrar data/calls/calls.jsonl e data/knowledge/call-learned.jsonl para Postgres (tabelas calls, call_learned).

npm run dev:web

Servidor com webhooks /api/calls/twilio/*

npx tsx src/integrations/call-system/CallSystemBridge.ts

Bridge standalone de chamadas

curl -X POST http://localhost:3000/api/calls/outbound -H 'Content-Type: application/json' -d '{"to":"+351..."}'

Testar outbound

%, Ficheiros: src/web/calls/{store,learning,routes}.ts, src/integrations/call-system/CallSystemBridge.ts, .env TWILIO_*.

3 Fase Voz (VoiceBridge)

- % MISSING: streaming de áudio WebSocket para o widget de voz no navegador (hoje é texto + resposta por turno).
- % MISSING: ligar as falas de Pedro/Trinnity (PT/EN/ES) a TTS real (ElevenLabs já configurado em CallSystemConfig).
- % MISSING: registos — data/voice/history.jsonl e data/knowledge/voice-learned.jsonl !' Postgres.

npm run dev

Core com VoiceBridge em localhost:3001 (report server)

cd src/dashboard && npm run dev

Dashboard com widget de voz (voice-widget.js)

- % Ficheiros: src/voice/VoiceBridge.ts, src/dashboard/public/voice-widget.js, src/dashboard/server.ts.

4 Fase Sites gerados (D3)

- % MISSING: deploy automático de cada site gerado para um domínio/pasta pública (hoje fica em data/sites/<slug> e /sites/<slug> no servidor).
- % MISSING: exportar ZIP de cada site (package: archiver) e endpoint /api/sites/:slug/download.
- % MISSING: vendê-lo — criar plano 'Sites' no billing (site + domínio + manutenção).

```
curl -X POST http://localhost:3000/api/sites/generate -H 'Content-Type: application/json' -d '{"name":"X","description":"Y"}'
```

Gerar site de teste

```
curl http://localhost:3000/api/sites/list
```

Listar sites gerados

- % Ficheiros: src/web/sites/{store,generator,routes}.ts.

5 Fase APKs gerados (D4)

- % MISSING: build real de cada scaffold — `cd data/apps/<slug> && npm i && npx expo prebuild && gradlew assembleRelease`.
- % MISSING: EAS cloud (npx eas login) para gerar APK/IPA sem SDK local e ligar /api/apps/:slug/build.
- % MISSING: registos — data/apps/apps.json !' Postgres + estado de build por app.

```
curl -X POST http://localhost:3000/api/apps/generate -H 'Content-Type: application/json' -d '{"name":"X","description":"Y"}
```

Gerar app de teste

```
curl http://localhost:3000/api/apps/myslug/source
```

Ver todos os ficheiros gerados

% Ficheiros: src/web/apps/{store,generator,routes}.ts.

6 Fase Negócio / atendimento (D5)

% MISSING: UI no dashboard — criar/editar agentes, ver histórico e KPIs de mensagens.

% MISSING: integração com email (responder clientes por Gmail) e com chamadas (D2).

% MISSING: billing por agente (plano novo no Avirato/Stripe) e limites por plano.

% MISSING: registos — data/business/agents.json + messages !' Postgres (tabelas business_agents, business_messages).

```
curl -X POST http://localhost:3000/api/business/agents -H 'Authorization: Bearer <token>' -H 'Content-Type: application/json' -d '{"name":"X","description":"Y"}
```

Criar agente (JWT)

```
curl -X POST http://localhost:3000/api/business/agents/<id>/messages -d '{"from":"cli","message":"oi"}
```

Cliente conversa

% Ficheiros: src/web/business/{store,routes}.ts.

7 Fase Produção / infraestrutura

% MISSING: Postgres — criar tabelas calls, call_learned, business_agents, business_messages, sites, apps (migrações em migrations/).

% MISSING: deploy automático no Render após cada push (hoje manual via API POST /services/{id}/deploys com body {}).

% MISSING: monitorização das novas APIs no /api/health e /api/metrics.

% MISSING: backup diário de data/calls, data/business, data/sites, data/apps (npm run backup).

```
npm run deploy
```

Deploy GitHub + Vercel (PDFs regenerados automaticamente)

```
node -e "fetch('https://api.render.com/v1/services/'+process.env.RENDER_SERVICE_ID+'/deploys',{method:'POST',headers:{Authorization:'Bearer '+process.env.RENDER_API_KEY,'Content-Type':'application/json'},body: '{}'}).then(r=>r.json()).then(console.log)"
```

Deploy forçado Render via API

npm run restart

Reinício à prova de congelamento (mata órfãos + verifica health/os/revenue)

npm run update:auto

Self-update completo: pull + install + PDFs + build + testes + deploy

npm run backup

Backup diário

%, Ficheiros: migrations/001_schema.sql, src/web/standalone-server.ts, scripts/restart.ps1, scripts/self-update.ps1.

8 Testes que faltam escrever

tests/web.test.ts

Adicionar blocos para /api/calls/*, /api/sites/*, /api/apps/*, /api/business/* (JWT + rate-limit)

tests/os.test.ts

Verificar que o TVS OS continua 25/25 após novas APIs

npm run test

Suíte completa antes de cada deploy